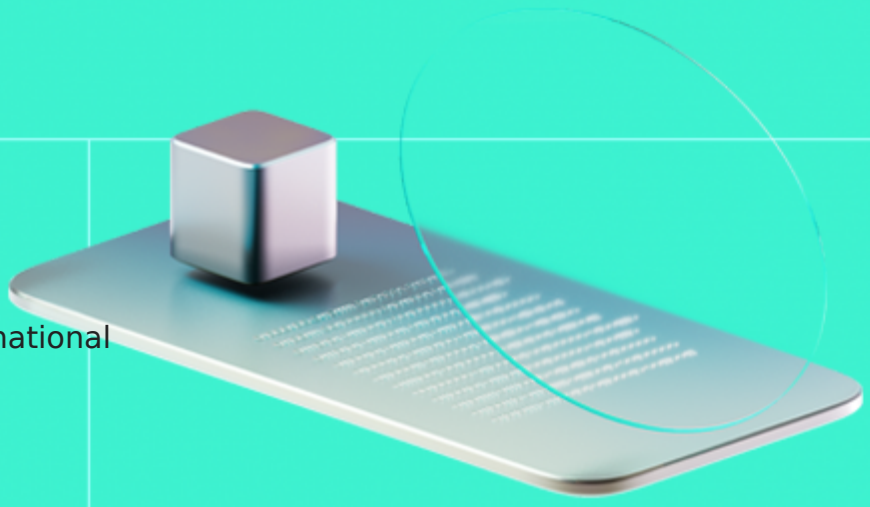
HACKEN

Smart Contract Code Review And Security Analysis Report



Customer: Digital Oro International

Date: 19/11/2025

We express our gratitude to the Digital Oro International team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

DOI - Digital Oro International. The best and easiest way to invest in real estate and make crypto income at the same time.

Document

Name	Smart Contract Code Review and Security Analysis Report for Digital Oro International
Audited By	David Camps Novi, Stepan Chekhovskoi
Approved By	Ivan Bondar
Website	https://doi.global
Changelog	10/10/2025 - Preliminary Report
	19/11/2025 - Final Report
Platform	Ethereum
Language	Solidity
Tags	ERC-721, Decentralized Finance, Staking.
Methodology	https://hackenio.cc/sc_methodology

Review Scope

Repository	https://github.com/digitaloro/sweepstakes
Initial Commit	727fbe1794d56377d594e11b99aa247aafa3da5d
Final Commit	17574745196752b1007f834d404486fc13200e7f

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

16	14	2	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	1
High	2
Medium	3
Low	3

Vulnerability	Severity
F-2025-13311 - Payout Claim Duration can be Surpassed	Critical
F-2025-13207 - User Funds Unexpectedly Transferred to Treasury due to Arbitrary Transfer From Parameter	High
F-2025-13424 - Payout Distribution Delays due to Invalid Last Claimed Timestamp Update	High
F-2025-13271 - Maximum Token Supply may Be Never Reached due to Invalid Validation	Medium
F-2025-13316 - Total Supply ERC-721 Enumeration Incompliance	Medium
F-2025-13317 - Excessive Token Mint due to Invalid Validation	Medium
F-2025-13238 - Token URI ERC-721 Metadata Incompliance	Low
F-2025-13391 - Raffle Participation Limits Admin Control of DOI Token Supply	Low
F-2025-13423 - Bypass of the Ownable2Step Transfer Mechanism	Low
F-2025-13243 - Invalid Hardcoded Token Naming	Info
F-2025-13247 - Unused Service Function	Info
F-2025-13381 - Incorrect NatSpec	Info
F-2025-13382 - Unused Parameter prizeContractAddress	Info
F-2025-13390 - Redundant Access Control Check	Info
F-2025-13420 - Inefficient Loop Due to Uncached Length Value	Info
F-2025-13425 - String Parameters in Custom Errors	Info

Documentation quality

- NatSpec comments are provided.
- Functional requirements are partially outdated.
- Technical description is provided.

Code quality

- The code architecture is clear.
- Few inconsistencies between similar codepieces are found.
- Some check duplications are identified.
- Several template code patterns are presented.
- The development environment is configured.
- Check-Effects-Interactions best-practice is not followed.

Test coverage

Code coverage of the project is **78%** (branch coverage).

- Deployment and basic user interactions are covered with tests.
- Some edge cases lack thorough testing.
- Several functions and branches in the `DOI_Token` contract are not covered with tests.

Table of Contents

System Overview

DOI - Digital Oro International is a set of smart contracts implementing the “DOI Token” and “DOI Gold” ERC-721 tokens. As well, the smart contracts include Staking, Raffle, and project Treasury functionalities.

- `DOI_AdminManager` manages a set of addresses considered as admins. This contract is part of an access control system where only the owner can modify the list of admins.
- `DOI_AccessControl` serves as an access control mechanism inherited by other contracts. The contract interacts with `DOI_AdminManager` to get the admins list.
- `DOI_Treasury` manages all the protocol funds. The contract implements methods for internal contracts to takes funds whenever needed. The owner is authorized to withdraw any funds.
- `DOI_PayoutManager` is a contract designed to manage and distribute scheduled payouts to the users.
- `DOI_Gold` is an ERC-721 with stakeholder privileges in the protocol. It is required to hold Gold tokens in order to stake in Real Estate.
- `DOI_Token` is an ERC-721 that can be used to participate in Raffle waves as well as buying Gold tokens.
- `DOI_RealEstate` is a representation of a real off-chain real estate asset. Gold holders can stake USDC in these contracts in order to get some yield.
- `DOI_LockManager` is a helper contract that manages token locks in the system, which may come from raffle prizes or real estate staking.
- `DOI_Referrals` introduces the referral feature of DOI Token purchases. When a user buys new tokens, if they were referred, a reward will be generated.
- `DOI_Raffle` contract implements a raffle system where users can participate using “DOI Token”. It features a wave-based structure where each wave allows a limited number of participants, and at the end of each wave, a winner for one time price or for a scheduled payout is selected.
- `DOI_RaffleVRF` wraps communication with Chainlink VRF provider to provide `DOI_Raffle` with on-chain randomness.

Privileged roles

The system defines several privileged roles with distinct responsibilities and access levels:

- **Owner:** The highest-privileged role, typically controlling the entire system. The owner can perform configuration operations such as assigning admin roles, updating Treasury accounting, or changing critical system addresses like the royalty recipient. The owner also inherits the permissions of admins, internal contracts, and approved contracts, allowing full control over all system functions.
- **Admin:** Assigned by the owner, admins can perform administrative tasks required for the system’s operation. This includes verifying Gold tokens, starting raffle waves, or pausing contracts.
- **Internal Contracts:** Defined by the owner, these are system contracts that require elevated privileges to execute core flows correctly, by enabling cross-calls between the

system contracts. Examples include initializing payouts, processing referral commissions, or updating user lockings in the Treasury. Internal contracts can also perform the operational permissions that approved contracts can.

- **Approved Contracts:** Assigned by the admin, approved contracts can interact with the Locking external functions, such as creating or withdrawing locks.

Potential Risks

- The project is fully centralized, introducing single points of failure and control. This centralization can lead to vulnerabilities in decision-making and operational processes, making the system more susceptible to targeted attacks or manipulation.
- The project iterates over large dynamic arrays, which leads to excessive gas costs, risking denial of service due to out-of-gas errors, directly impacting contract usability and reliability.
- The system Staking and Raffle distribute funds from Treasury. Initially, the Treasury is filled with funds received from “DOI Token” sale. Treasury is supposed to be funded externally to cover the payouts.
- The `DOI_Treasury` contract is the sole source of funds for raffle, real estate, and referral rewards, but it has no on-chain mechanism to autonomously generate or secure these funds. Its balance depends entirely on manual deposits by system administrators, introducing centralization and trust assumptions. If mismanaged or underfunded, users may be unable to claim their expected rewards.
- The system’s access control relies on the `onlyAuthorizedContract` and `onlyInternalContract` modifiers, which depend on `approvedContracts` and `internalContracts` mappings managed by the admin or owner roles. Since these roles are centrally controlled, key functions—such as processing referral commissions, withdrawing locked funds or altering user balances in treasury—ultimately depend on trusted administrative management. This introduces a centralization risk, as misuse or misconfiguration of these roles could compromise the fairness or integrity of the system’s operations.
- `DOI_Gold` ERC721 tokens serve as eligibility assets for participating in real estate staking and acting as verified stakeholders within the system. However, the verification process is performed off-chain by the system administrator, who must manually trigger the on-chain `verifyFunction()` afterward. This introduces a trust dependency on the admin’s off-chain actions, as users cannot independently verify or trigger their own verification, making the process centralized and dependent on proper administrative execution.
- In the `replaceToken()` function, the admin has the ability to burn a user’s existing Gold token and mint a new one to a different address. This effectively allows the admin to revoke ownership of tokens without the holder’s consent. Such unrestricted administrative control introduces a centralization risk and undermines token ownership guarantees, as users must fully trust the admin not to misuse this capability.
- The system accepts token amounts as human-readable values (without decimals) in several functions, relying on internal scaling logic to maintain consistency across the execution flow. As a result, fractional token amounts cannot be used. Moreover, if a system manually triggers functions outside the intended internal call sequence—bypassing the approved execution paths—accounting inconsistencies may occur, potentially leading to inaccurate balances or other unintended behaviors.
- The `PayoutManager` contract calculates payouts by dividing the total payout amount by the number of unlock periods (`payoutPerPeriod = totalPayout / totalPeriods`). Due to integer truncation in Solidity, the cumulative sum of all periodic payouts (`payoutPerPeriod * totalPeriods`) may be slightly lower than the intended total payout. Consequently, users may receive marginally fewer tokens than expected. While the impact on users is minor given the small rounding discrepancy, administrators should monitor the Treasury’s

balance to ensure accounting consistency. Any surplus tokens allocated to the Treasury to offset such discrepancies can be withdrawn by authorized roles, mitigating potential operational concerns.

- Each `RealEstate` contract defines a maximum total amount of USDC that can be staked; however, no per-user staking limit is enforced. As a result, a single user could potentially stake the entire available capacity of a given `RealEstate` contract, preventing other participants from taking part in the staking opportunity and undermining fairness in access to the system's rewards.
- The protocol's revenue, which funds RealEstate staking rewards, raffle prizes, and referral rewards, appears to be generated off-chain—presumably from activities such as selling or renting properties. There is no on-chain mechanism to verify or enforce these revenue inflows, meaning the system relies entirely on administrators or owners to accurately and fairly update the Treasury. This introduces a centralization and trust risk, as users cannot independently verify the source or correctness of funds backing the rewards, and mismanagement could result in insufficient payouts.
- The `Treasury` contract holds users' locked USDC tokens from RealEstate staking, as well as reward tokens, with the locked funds being the most critical. The contract exposes `withdraw()` and `withdrawTo()` functions, which grant the owner the ability to transfer out these funds. This introduces a centralization and trust risk, as the contract owner could potentially drain user deposits, compromising the safety of staked assets and overall system integrity.
- Normally, users are required to hold at least one Gold token to maintain their active locks, and the `_update()` function in the Gold contract enforces this by reverting transfers that would leave a user without any Gold tokens. However, the `replaceToken()` function bypasses this safeguard, allowing a user to hold active locks without possessing any Gold tokens.
- When the Gold token reaches its supply limit of 5,000 tokens, the `DOI_Raffle` contract allows an active raffle to be closed even if users have already participated. In this situation, a winner is still selected; however, the expected on-chain payout mechanisms (`doiTreasury.processPayment()` or `payoutManager.initializePayout()`) are not executed. Instead the winner will receives a reduced prize, managed off-chain. This discrepancy between expected and actual payout behavior may lead to user confusion or disputes. The conditions under which a raffle may close with modified prize distribution should be clearly disclosed within the raffle rules.
- The system relies on multiple cross-contract interactions between `DOI_Raffle`, the `DOI_LockManager` contract, or the `DOI_Treasury` contract, among others. Certain functions involve external calls before all internal state changes are finalized, which deviate from the Checks-Effects-Interactions (CEI) pattern. While no direct exploit or reentrancy pathway was identified, inconsistent adherence to CEI can increase the risk of unexpected state dependencies, partial execution, or unforeseen side effects during cross-contract calls. Ensuring strict CEI compliance across all interacting components is recommended to maintain predictable and secure execution flows.

Findings

Vulnerability Details

F-2025-13311 - Payout Claim Duration can be Surpassed - Critical

Description:

The method `claimUserPayout()` is used to calculate and transfer a user's available payout for a given id. The function checks how much time passed since the previous claim, and use it to calculate the claimable amount.

```
periodsPassed = (currentTime - payout.lastPayoutTime) / periodInSeconds;
uint256 totalAmount = payout.amount * periodsPassed;
doiTreasury.processPayment(payout.beneficiary, totalAmount);
```

Each payout has a defined `duration` configured at the moment of creation, corresponding to the number of periods the payout should last. Once the duration is reached, the payout is deactivated and removed from the contract.

```
if (payout.claimedPeriods >= payout.duration) {
    // Mark payout as completed and move to completed status
    payout.isActive = false;
    activePayouts.remove(payoutId);
    activeUserPayouts[payout.beneficiary].remove(payoutId);
    userPayouts[payout.beneficiary].push(payoutId);

    ...
}
```

However, there is no verification that ensures the `periodsPassed` cannot surpass the remaining periods.

As a consequence, if the elapsed time since the last claim is large enough, `periodsPassed` may exceed the remaining duration, allowing the caller to claim more tokens than intended and leading to potential overpayment and premature depletion of treasury funds.

Assets:

- contracts/DOI_PayoutManager.sol [https://github.com/digitaloro/sweepstakes]

Status: Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Critical

Recommendations

Remediation: Consider adding a check in `claimUserPayout()` to limit the `periodsPassed` to the remaining periods:

```
uint256 periodsRemaining = payout.duration - payout.claimedPeriods;
periodsPassed = periodsPassed > periodsRemaining ? periodsRemaining : periods
Passed;

uint256 totalAmount = payout.amount * periodsPassed;
```

Resolution: The Finding is fixed in the commit `2a5f4e4`.

The `periodsPassed` is limited to `payout.duration - payout.claimedPeriods` at max.

Evidences

Proof of Concept

Reproduce: As it can be seen in the foundry test example below, a payout is created for 7 day duration and 1_000 tokens per period, corresponding to a total amount of 7_000 tokens. However the caller is able to release 10_000 since it's calling the function 10 days after creation of the payout.

```
function test_poc_payout_duration_overcome() public {
    address beneficiary = makeAddr("beneficiary");
    uint256 payoutAmountPerPeriod = 1_000;
    DOI_PayoutManager.PeriodType periodType = DOI_PayoutManager.PeriodType.DA
Y;
    uint256 duration = 7;
    string memory info = "";
    uint256 initialTimestamp = block.timestamp;
```

```
    payoutManager.setInternalContract(address(this));
    uint256 payoutId = payoutManager.initializePayout(beneficiary, payoutAmou
ntPerPeriod, periodType, duration, info);
    vm.warp(initialTimestamp + 10 days);

    //transfer tokens to treasury to allow claim
    usdt.transfer(address(treasury), usdt.balanceOf(address(this)));

    uint256 initialBeneficiaryBalance = usdt.balanceOf(beneficiary);
    payoutManager.claimUserPayout(payoutId);
    uint256 finalBeneficiaryBalance = usdt.balanceOf(beneficiary);

    uint256 receivedTokens = finalBeneficiaryBalance - initialBeneficiaryBala
nce;
    uint256 totalPayoutAmount = payoutAmountPerPeriod * duration;
    assert(receivedTokens > payoutAmountPerPeriod);

    console.log("received tokens", receivedTokens);
    console.log("total payout amount", totalPayoutAmount);
}
```

Results:

```
[PASS] test_poc_payout_duration_overcome() (gas: 396190)
Logs:
  received tokens 10000
  total payout amount 7000

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 15.02ms (2.99ms
CPU time)
```

F-2025-13207 - User Funds Unexpectedly Transferred to Treasury due to Arbitrary Transfer From Parameter - High

Description:

The `receivePayment` function of the `DOI_Treasury` contract is part of the payment system, allowing anyone to deposit funds directly into the Treasury as a form of service payment.

However, the function calls `token.safeTransferFrom` with an arbitrary `from` parameter provided by the caller. This allows anyone to drain user allowances by transferring user-approved funds to the Treasury.

Users are expected to grant allowances to the Treasury in order to deposit funds and participate in the Locks functionality. As a result, there may be multiple unspent allowances left on the Treasury contract, with new allowances being granted on a regular basis.

The `receivePayment` function does not update the internal Treasury user balance. This is intended by design as services payment is not expected to be deposited to internal Treasury user balance.

Any funds deposited into the Treasury are recoverable by the contract owner. The `withdrawTo` function allows the owner to manually return funds to users.

```
function receivePayment(
    address user,
    uint256 amount,
    string memory service
) external nonReentrant {
    ...
    usdtToken.safeTransferFrom(user, address(this), amount);
    ...
}

function withdrawTo(uint256 amount, address recipient) external onlyOwner non
Reentrant {
    ...
    usdtToken.safeTransfer(recipient, amount);
    ...
}
```

This may allow a malefactor to drain existing user allowances and front-run user deposits, transferring user funds to the Treasury without increasing their internal Treasury balance. This breaks the intended Treasury flow and forces the contract owner to manually recover all affected user funds.

Assets:

- contracts/DOI_Treasury.sol [https://github.com/digitaloro/sweepstakes]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: High

Recommendations

Remediation: Consider changing the `receivePayment` function to only process caller funds:

```
function receivePayment(uint256 amount, string memory service) external nonRe
entrant {
    ...
    usdtToken.safeTransferFrom(msg.sender, address(this), amount);
    ...
}
```

The `receivePayment` function is used in the `DOI_Token` contract to process user payments. Consider changing the design so that users approve funds for the `DOI_Token` contract, which would then transfer the funds from the user to the Treasury:

```
function _purchaseTokensWithUSDT(uint256 tokensToMint, address referrer) inte
rnal returns (bool) {
    ...
    // doiTreasury.receivePayment(msg.sender, totalToPay, "token_purchase");
    usdtToken.safeTransferFrom(msg.sender, address(doiTreasury), totalToPay);
    ...
}
```

Resolution: The Finding is fixed in the commit `2a5f4e4`.

The `receivePayment` function is limited to process only user funds. The `_purchaseTokensWithUSDT` directly transfers funds from the user to the

Treasury.

Evidences

PoC

Reproduce:

```
it("Hacken: User Treasury Allowance Drain by 3rd Party", async function () {
    const [, , , , , user, malefactor] = await ethers.getSigners();

    const allowance = 2n ** 256n - 1n;
    const balance = 50000n * 10n ** 6n;

    await usdtToken.connect(user).mintTestnetTokens();

    // User approves funds
    await usdtToken.connect(user).approve(treasury.getAddress(), allowance);

    // Malefactor drains user allowance
    await treasury.connect(malefactor).receivePayment(user.getAddress(), bala
nce, "");

    // User funds consistency broken
    expect(
        (await treasury.userBalances(user.getAddress())) + (await usdtToken.b
alanceOf(user.getAddress()))
    ).to.equal(balance);
});
```

Results:

```
Hacken: User Treasury Allowance Drain by 3rd Party:

  AssertionError: expected 0 to equal 50000000000.
  + expected - actual

  -0
  +50000000000
```

F-2025-13424 - Payout Distribution Delays due to Invalid Last Claimed Timestamp Update - High

Description:

The `claimUserPayout` function of the `DOI_PayoutManager` contract aims to distribute payout funds proportionally to the number of periods passed. The number of periods passed is calculated as follows.

```
uint256 periodsPassed = (currentTime - payout.lastPayoutTime) / periodInSeconds
```

The `lastPayoutTime` is updated to the current timestamp.

```
uint256 currentTime = block.timestamp;
payout.lastPayoutTime = currentTime;
```

Such an approach ignores current distribution period is in progress and shifts the distribution schedule delaying the payments.

Assets:

- contracts/DOI_PayoutManager.sol [https://github.com/digitaloro/sweepstakes]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: High

Recommendations

Remediation:

Consider increasing the `lastPayoutTime` by the `periodsPassed * periodInSeconds` portion instead of complete reassignment.

```
payout.lastPayoutTime += periodsPassed * periodInSeconds;
```

Resolution: The Finding is fixed in the commits `2a5f4e4` and `8190ccc`.

The `lastPayoutTime` state variable is updated according to the number of periods passed.

Evidences

PoC

Reproduce:

```
it("Hacken: Payout Delays", async function () {
  const treasuryAmount = ethers.parseUnits("100000", 6);
  await usdtToken.mint(await treasury.getAddress(), treasuryAmount);

  const amount = 1000 * 10 ** 6; // 1000 USD per month (scaled)
  const periodType = 2; // MONTH
  const duration = 12;
  const info = "Prize for winning raffle";

  // Initialize the payout
  await payoutManager.connect(owner).initializePayout(user.address, amount, p
eriodType, duration, info);

  // Verify the payout was initialized correctly
  const activeUserPayouts = await payoutManager.getUserActivePayouts(user.add
ress);
  expect(activeUserPayouts[0].amount).to.equal(amount);
  expect(activeUserPayouts[0].isActive).to.be.true;

  // Simulate time passing (1.6 month)
  await time.increase(time.duration.days(48));

  const initialUserBalance = await usdtToken.balanceOf(user.address);

  // Claim the payout
  await payoutManager.connect(user).claimUserPayout(0);

  const middleUserBalance = await usdtToken.balanceOf(user.address);

  // Simulate time passing (1.6 month)
  await time.increase(time.duration.days(48));

  // Claim the payout
  await payoutManager.connect(user).claimUserPayout(0);

  const finalUserBalance = await usdtToken.balanceOf(user.address);
```

```
  console.log("While in total over 3 month passed, only 2 payments released")
;


  console.log(initialUserBalance);
  console.log(middleUserBalance);
  console.log(finalUserBalance);
});
```

Results:

```
  PayoutManager
    Payout Claiming
While in total over 3 month passed, only 2 payments released
0n
1000000000n
2000000000n
      ✔ Hacken: Payout Delays
```

F-2025-13271 - Maximum Token Supply may Be Never Reached due to Invalid Validation - Medium

Description:

The `mintGoldMembership` function of the `DOI_Gold` contract is expected to allow users to mint new tokens untill the `MAX_SUPPLY` constant is reached.

```
function mintGoldMembership() public nonReentrant {
    uint256 currentSupply = tokenCounter;
    if (currentSupply > MAX_SUPPLY) revert TotalSupplyLimitReached();

    ...

    uint256 newSupply = currentSupply + 1;
    tokenCounter = newSupply;  // Update in storage

    // Mint the Gold Membership token
    _safeMint(msg.sender, newSupply);

    ...
}
```

The `limitReached` view function is expected to return if the supply limit is reached.

```
function limitReached() external view returns (bool) {
    return tokenCounter >= MAX_SUPPLY;
}
```

However, the functions use the `tokenCounter` variable as the current token supply value. This variable represents an increasing counter to be used as token id for new tokens minted and does not track the number of tokens in the contract (F-2025-13316).

In fact, the `tokenCounter` variable value may exceed the actual current token supply, in case the `replaceToken` function (admin restricted) was ever called.

```
function replaceToken(
    uint256 oldTokenId,
    address recipient,
    string calldata reason
) external onlyAdmin {
    ...
```

```
    // Burn the old token
    _burn(oldTokenId);

    // Mint the new token
    uint256 newTokenId = tokenCounter + 1;
    tokenCounter = newTokenId;
    _safeMint(recipient, newTokenId);

    ...
}
```

This may lead to `mintGoldMembership` function not allowing token minting while the actual token supply is below the `MAX_SUPPLY` constant.

Assets:

- contracts/DOI_Gold.sol [https://github.com/digitaloro/sweepstakes]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 5/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Consider removing the custom `totalSupply` function implementations (F-2025-13316) and using the function inherited from OpenZeppelin to identify the current token supply.

```
function mintGoldMembership() public nonReentrant {
    uint256 currentSupply = totalSupply();

    ...

    uint256 newTokenId = tokenCounter + 1;
    tokenCounter = newTokenId;
    _safeMint(msg.sender, newTokenId);
```

```
    ...
}

function limitReached() external view returns (bool) {
    return totalSupply() >= MAX_SUPPLY;
}
```

Resolution: Fixed in commit ID `6a67ac3`: the custom `totalSupply()` was removed from the code, using the parent's contract function in order to check the amount of existing token for sensitive operations, such as `limitReached()`.

F-2025-13316 - Total Supply ERC-721 Enumeration Incompliance - Medium

Description:

The `DOI_Gold` and `DOI_Token` contracts are ERC-721 token implementations. The contracts declare `totalSupply` function which according to EIP-721 standard is meant to return the number of valid tokens tracked by the contract.

```
function totalSupply() public view override returns (uint256) {
    return tokenCounter;
}
```

However, the `tokenCounter` variable (used as return value of the `totalSupply` function) represents increasing counter to be used as token id for new tokens minted and does not consider token burn possibility.

The `replaceToken` function (admin restricted) of the `DOI_Gold` contract burns token and mints a new one with a new token id, causing the `tokenCounter` increase while the actual number of tokens is kept same.

```
function replaceToken(
    uint256 oldTokenId,
    address recipient,
    string calldata reason
) external onlyAdmin {
    ...

    // Burn the old token
    _burn(oldTokenId);

    // Mint the new token
    uint256 newTokenId = tokenCounter + 1;
    tokenCounter = newTokenId;
    _safeMint(recipient, newTokenId);

    ...
}
```

This may lead to the `totalSupply` function return an invalid value potentially causing issues in external services.

Assets:

- contracts/DOI_Token.sol [https://github.com/digitaloro/sweepstakes]

- contracts/DOI_Gold.sol [https://github.com/digitaloro/sweepstakes]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 5/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: The `DOI_Gold` and `DOI_Token` contracts inherit `totalSupply` function declaration from the `ERC721Enumerable` OpenZeppelin implementation. Consider removing the custom `totalSupply` function declarations from the contracts.

Resolution: The Finding is fixed in the commit `c19b227`.

The custom `totalSupply` function declarations are removed.

F-2025-13317 - Excessive Token Mint due to Invalid Validation - Medium

Description:

The `mintGoldMembership` function of the `DOI_Gold` contract is expected to allow users to mint the `MAX_SUPPLY` of tokens.

```
function mintGoldMembership() public nonReentrant {
    ...

    if (currentSupply > MAX_SUPPLY) revert TotalSupplyLimitReached();

    ...
}
```

However, the validation allows minting when the `currentSupply` equals to `MAX_SUPPLY` meaning one excessive token can be minted.

This may lead to the total supply exceeds the `MAX_SUPPLY` constant.

Assets:

- contracts/DOI_Gold.sol [https://github.com/digitaloro/sweepstakes]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation:

Consider updating the check to ensure that the current supply is below the `MAX_SUPPLY` constant.

```
if (currentSupply >= MAX_SUPPLY) revert TotalSupplyLimitReached();
```

Resolution: The Finding is fixed in the commit `c19b227`.

The check is changed to prevent excessive token minting.

F-2025-13238 - Token URI ERC-721 Metadata Incompliance - Low

Description:

The `DOI_Gold` and `DOI_Token` contracts are ERC-721 token implementations. The contracts declare `tokenURI` function meant to return the metadata URI for an NFT by the token id.

According to EIP-721 standard, the function should throw an error, in case specified token id is invalid. However, instead of reverting, the function returns same URL string violating the standard.

This may lead to various external services incorrectly process the token existence checks.

```
function tokenURI(uint256 tokenId) public pure override returns (string memor
y) {
    ...
    return "https://doi.global/doi-gold.json";
}

function tokenURI(uint256 tokenId) public pure override returns (string memor
y) {
    ...
    return "https://doi.global/doi-token.json";
}
```

Assets:

- contracts/DOI_Token.sol [https://github.com/digitaloro/sweepstakes]
- contracts/DOI_Gold.sol [https://github.com/digitaloro/sweepstakes]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Consider implementing the token id validation.

For example, the standard implementation of ERC-721 from OZ provides a check for the presence of the owner available in the contract by inheritance.

```
function tokenURI(uint256 tokenId) public view override returns (string memor
y) {
    _requireOwned(tokenId);
    ...
}
```

Resolution: The Finding is fixed in the commit `c19b227`.

The `tokenURI` functions check for tokens existence.

F-2025-13391 - Raffle Participation Limits Admin Control of DOI Token Supply - Low

Description:

In the `DOI_Token` contract, minting can be disabled via two functions: `disableMintingOnLastSweepstakesEnd()` and `disableMintingOnGoldMembershipReached()`. Both functions perform the following check before disabling minting:

```
if (!doiRaffle.hasActiveWave()) {
    mintingDisabled = true;
    emit MintingDisabled();
}
```

This means that minting can only be stopped when there are no active raffle waves. However, raffle waves in the `DOI_Raffle` contract cannot be closed until the participants limit is reached in `drawWinner()`:

```
if (entries.length < _participantLimit) {
    revert ParticipantLimitNotReached();
}
```

As a result, the deactivation of DOI token minting is dependent on user participation. If token holders do not participate sufficiently—especially in the wave coinciding with the 5,000 gold token supply being reached or the last sweepstakes—minting cannot be disabled. This makes the timing of minting deactivation partially out of the control of system administrators and introduces a risk that the DOI token supply could exceed intended limits, potentially causing unintended inflation or undesired tokenomics behavior.

Assets:

- contracts/DOI_Raffle.sol [https://github.com/digitaloro/sweepstakes]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Consider making the call to `drawWinner()` more flexible, so that raffle waves can be closed when the gold supply reaches `5000` tokens as an alternative to reaching the `_participantsLimit`.

```
if (entries.length < _participantLimit && !doiGold.limitReached()) {
    revert ParticipantLimitNotReached();
}
```

Resolution: Fixed in commit ID `42a127b`: the DOI token minting was disconnected from the active raffle waves. Now, the token minting is disabled automatically when the Gold token supply reaches `5000` by triggering `disableMintingOnGoldMembershipReached()` in `mintGoldMembership()`:

```
function disableMintingOnGoldMembershipReached()
    external
    onlyGoldMemberContract
{
    if (!mintingDisabled) {
        mintingDisabled = true;

        emit MintingDisabled();
    }
}
```

Additionally, the `DOI_Raffle` methods `drawWinner()` and `finalizeWave()` were updated to support the closure of waves when gold membership limit is reached.

F-2025-13423 - Bypass of the Ownable2Step Transfer Mechanism - Low

Description:

The `transferToMultisig()` function bypasses the two-step ownership transfer mechanism provided by OpenZeppelin's `Ownable2Step` by directly calling the internal `_transferOwnership()` function. This allows the owner to transfer ownership immediately without the usual confirmation step, reducing the safety guarantees intended by the two-step process and potentially increasing the risk of accidental or unauthorized ownership transfers.

```
function transferToMultisig(address multisigSafe) external onlyOwner {
    if (multisigSafe == address(0)) {
        revert ZeroAddressNotAllowed(bytes32("MultisigSafe"));
    }
    _transferOwnership(multisigSafe);
}

function _transferOwnership(address newOwner) internal virtual override {
    delete _pendingOwner;
    super._transferOwnership(newOwner);
}

function _transferOwnership(address newOwner) internal virtual {
    address oldOwner = _owner;
    _owner = newOwner;
    emit OwnershipTransferred(oldOwner, newOwner);
}
```

Assets:

- contracts/DOI_Treasury.sol [https://github.com/digitaloro/sweepstakes]

Status: Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: It is recommended to use the `Ownable2Step` safer method `transferOwnership()` instead of `_transferOwnership()`:

```
function transferOwnership(address newOwner) public virtual override onlyOwne
r {
    _pendingOwner = newOwner;
    emit OwnershipTransferStarted(owner(), newOwner);
}
```

Resolution: Fixed in commit ID `42a127b`: the `transferOwnerhip()` function was introduced as recommended.

```
    function transferToMultisig(address multisigSafe) external onlyOwner {
        if (multisigSafe == address(0)) revert ZeroAddressNotAllowed();
        transferOwnership(multisigSafe);
    }
```

F-2025-13243 - Invalid Hardcoded Token Naming - Info

Description: There are multiple mentions of USDT stablecoin in comments and variable names within the codebase. According to the information from the Client, USDC stablecoin is going to be used instead.

For example:

```
/**
 * @dev Returns the total USDT balance held by this treasury contract.
 * @return The total USDT balance.
 */
function getUSDTBalance() public view returns (uint256) {
    return usdtToken.balanceOf(address(this));
}
```

Such inconsistency decreases code clarity.

Assets:

- contracts/DOI_Token.sol [https://github.com/digitaloro/sweepstakes]
- contracts/DOI_Raffle.sol [https://github.com/digitaloro/sweepstakes]
- contracts/DOI_LockManager.sol [https://github.com/digitaloro/sweepstakes]
- contracts/DOI_RealEstate.sol [https://github.com/digitaloro/sweepstakes]
- contracts/DOI_PayoutManager.sol [https://github.com/digitaloro/sweepstakes]
- contracts/DOI_Treasury.sol [https://github.com/digitaloro/sweepstakes]

Status: Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider changing the names for more abstract, e.g. `paymentToken`.

Resolution: The Finding is accepted by the Client team.

F-2025-13247 - Unused Service Function - Info

Description: The `min` function of the `DOI_PayoutManager` contract is supposed to be service function, however, is never used.

```
function min(uint256 a, uint256 b) internal pure returns (uint256) {
    return a < b ? a : b;
}
```

Unused code leads to higher deployment costs.

Assets:

- contracts/DOI_PayoutManager.sol [https://github.com/digitaloro/sweepstakes]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider removing the unused function.

Resolution: The Finding is fixed in the commit `c19b227`.

The function is removed.

F-2025-13381 - Incorrect NatSpec - Info

Description: The NatSpec for the `disableMinting()` function incorrectly states that the action is irreversible, while the `reEnableMinting()` function allows minting to be resumed.

```
@dev This function is irreversible and will prevent any new tokens from being minted.
```

```
function disableMinting() external onlyOwner {
    if (mintingDisabled) revert MintingAlreadyDisabled();
    mintingDisabled = true;
    emit MintingDisabled();
}

function reEnableMinting() external onlyOwner {
    if (!mintingDisabled) revert MintingAlreadyEnabled();
    if (doiGold.limitReached()) revert GoldMembershipLimitReached();
    mintingDisabled = false;
    emit MintingReEnabled();
}
```

This inconsistency between documentation and implementation can mislead developers, auditors, or users regarding the contract's behavior, potentially causing incorrect assumptions or integrations that rely on minting being permanently disabled.

Assets:

- contracts/DOI_Token.sol [https://github.com/digitaloro/sweepstakes]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider updating the NatSpec to reflect the actual behavior of the contract.

Resolution: Fixed in commit ID `42a127b`: the NatSpec of the `disableMinting()` method was updated to match its actual behavior, as seen below.

```
//@dev This function is disabled if the gold membership limit is reached.
```

F-2025-13382 - Unused Parameter prizeContractAddress - Info

Description: The `prizeContractAddress` variable is defined in the `DOI_Raffle` contract and included in both the `PrizeInfo` struct and the `WaveEnded` event through the `startNewWave()` and `finalizeWave()` functions. However, it is never used in the contract's logic. This makes the variable redundant, increasing code complexity and potential confusion for future maintainers or integrators. It should be removed to improve clarity and maintainability.

```
struct PrizeInfo {
    PrizeType prizeType;
    uint256 prizeAmount;
    uint256 prizeDuration;
    DOI_PayoutManager.PeriodType prizePeriodType;
    address prizeContractAddress;
}
```

Assets:

- contracts/DOI_Raffle.sol [https://github.com/digitaloro/sweepstakes]

Status: Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: It is recommended to remove unused parameters from the contracts.

Resolution: The Finding is accepted with a note:

> This value is used in the client app when retrieving the prize.

F-2025-13390 - Redundant Access Control Check - Info

Description: The `onlyRaffle` modifier in `DOI_RaffleVRF` contains a redundant access control check:

```
modifier onlyRaffle() {
    if (msg.sender != doiRaffleAddress) {
        if (msg.sender != doiRaffleAddress) revert NotAuthorized(); // Ensure
the caller is DOI_Raffle contract
    }
    _;
}
```

The inner `if` statement repeats the exact same condition as the outer `if`, making it unnecessary. This redundancy increases code complexity and may confuse readers.

Similarly, the `onlyAdmin` modifier in the `DOI_AccessControl` contract checks the `owner()` twice:

```
modifier onlyAdmin() {
    if (msg.sender != owner()) {
        if (!adminManager.isAdmin(msg.sender) && msg.sender != owner()) {/
            revert NotAuthorized();
        } // Ensure the caller is an admin
       }
    _;
}
```

Assets:

- contracts/DOI_RaffleVRF.sol [https://github.com/digitaloro/sweepstakes]
- contracts/DOI_AccessControl.sol [https://github.com/digitaloro/sweepstakes]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Reduce the redundant checks to a single one:

```
modifier onlyRaffle() {
    if (msg.sender != doiRaffleAddress) revert NotAuthorized();
    _;
}


modifier onlyAdmin() {
    if (!adminManager.isAdmin(msg.sender) && msg.sender != owner()) {
        revert NotAuthorized();
    } // Ensure the caller is an admin
    _;
}
```

Resolution: The Finding is fixed in the commit `c19b227`.

The modifiers are simplified to avoid duplications.

F-2025-13420 - Inefficient Loop Due to Uncached Length Value - Info

Description:

The `getUserActiveLocks()` function iterates through a dynamic `EnumerableSet` while repeatedly calling `active.length()` within the loop condition:

```
function getUserActiveLocks(address user)
    external
    view
    returns (Lock[] memory)
{
    EnumerableSet.UintSet storage active = activeUserLocks[user];
    Lock[] memory locks = new Lock[](active.length());
    for (uint256 i = 0; i < active.length(); i++) { // repeated length() call
s
        locks[i] = allLocks[active.at(i)];
    }
    return locks;
}
```

Although this function is a `view` function, it is not purely external — it is invoked internally by the `DOI_Gold` contract's `_update()` function to determine whether the sender has active locks. Repeatedly calling `active.length()` in each loop iteration introduces unnecessary overhead and slightly increases gas consumption for every invocation.

Caching the length in a local variable before the loop would make the code more efficient and reduce redundant reads from storage-backed structures like `EnumerableSet`.

Assets:

- contracts/DOI_LockManager.sol [https://github.com/digitaloro/sweepstakes]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Cache the `active.length()` value in a local variable before entering the loop:

```
uint256 activeLength = active.length();
for (uint256 i = 0; i < activeLength; i++) {
    locks[i] = allLocks[active.at(i)];
}
```

Resolution: The Finding is fixed in the commit `c19b227`.

The caching mechanism is implemented.

F-2025-13425 - String Parameters in Custom Errors - Info

Description: Custom Errors are designed to avoid usage of revert strings which increase code size and the deployment costs.

However the `InvalidAddress(string field)`, `MustBeGreaterThanZero(string field)` errors directly use string as an error parameter. The `ZeroAddressNotAllowed(bytes32 _contract)` error uses `bytes32` parameter which is assigned a `bytes32` wrapped string in most cases.

Such an approach is considered to be inefficient Custom Errors usage increasing the code size and the deployment costs.

Assets:

- contracts/DOI_CustomErrors.sol [https://github.com/digitaloro/sweepstakes]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider using different errors to distinguish failure reasons and provide meaningful execution dependent parameters for the errors dispatch.

Resolution: The Finding is mostly fixed in the commit `c19b227`.

Most of the inefficiencies are eliminated, however, the `TokenAlreadyUsed(string usage)` and `TicketAlreadyUsed(string name)` errors still use strings as parameters.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken’s ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.
The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at https://hacken.io/audits/.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

hknio/severity-formula

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project’s performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/digitaloro/sweepstakes
Initial Commit	727fbe1794d56377d594e11b99aa247aafa3da5d
Final Commit	17574745196752b1007f834d404486fc13200e7f
Whitepaper	N/A
Requirements	README.md
Technical Requirements	docs/*.md

Asset	Type
contracts/DOI_AccessControl.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
contracts/DOI_AdminManager.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
contracts/DOI_CustomErrors.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
contracts/DOI_Gold.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
contracts/DOI_LockManager.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
contracts/DOI_PayoutManager.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
contracts/DOI_Raffle.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
contracts/DOI_RaffleVRF.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
contracts/DOI_RealEstate.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
contracts/DOI_Referrals.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
contracts/DOI_Token.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
contracts/DOI_Treasury.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.